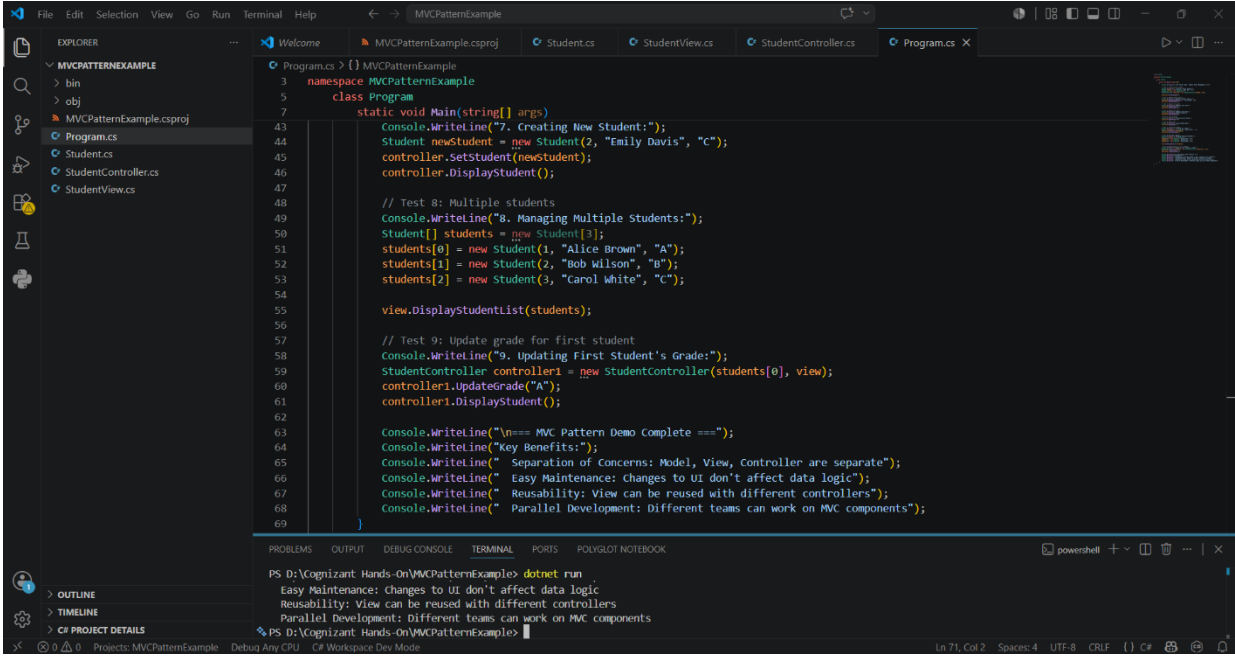


## Hands-On

### Exercise 10: Implementing the MVC Pattern

I have implemented, executed and tested the Model-View-Controller (MVC) Pattern in .NET, via a project called MVCPatternExample. I created a Student class to hold the Student's information (name, student ID, grade, etc.), a StudentView class to show the student information, and a StudentController class to handle the communication between the model and the view. The controller was responsible for retrieving, updating, and passing data between the Student class and the StudentView class. Once implemented, I ran the application and added and edited student records through the controller and then showed the updated student records in the view. This hands-on experience gave me a better understanding of how the MVC Pattern separates data, business logic and presentation layers, and how it enables .NET applications to be more structured, organized, and easier to maintain, scale and manage.

### Coding:

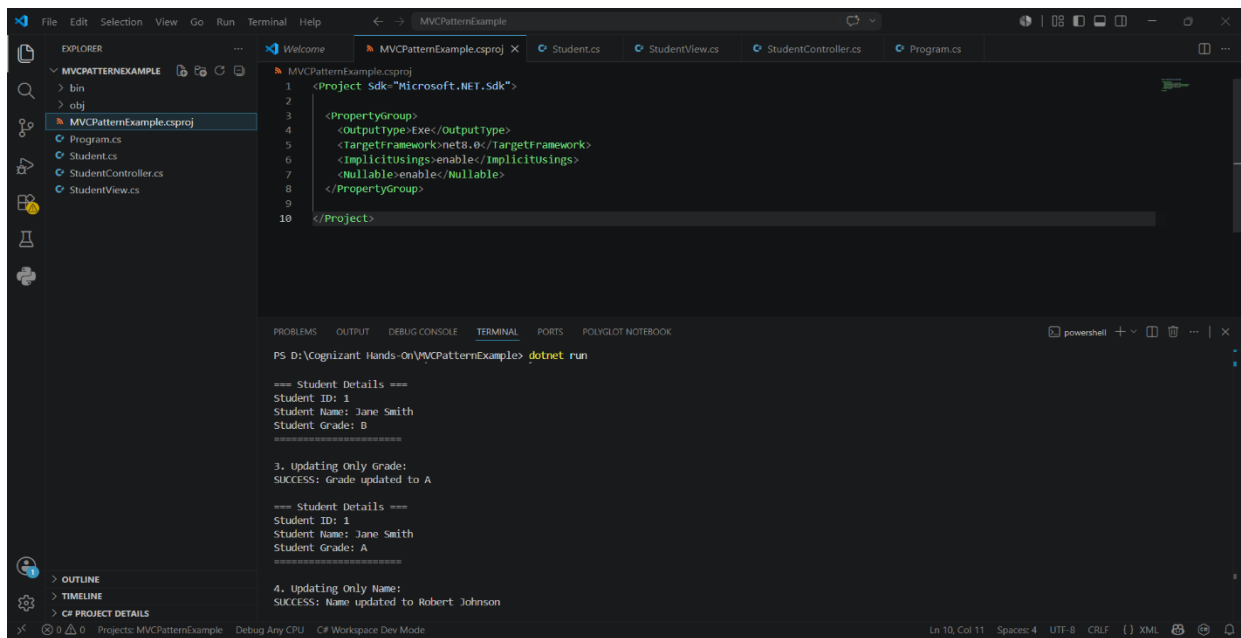


```
1 namespace MVCPatternExample
2 {
3     class Program
4     {
5         static void Main(string[] args)
6         {
7             Console.WriteLine("7. Creating New Student:");
8             Student newStudent = new Student(2, "Emily Davis", "C");
9             controller.SetStudent(newStudent);
10            controller.DisplayStudent();
11
12            // Test 8: Multiple Students
13            Console.WriteLine("8. Managing Multiple Students:");
14            Student[] students = new Student[3];
15            students[0] = new Student(1, "Alice Brown", "A");
16            students[1] = new Student(2, "Bob Wilson", "B");
17            students[2] = new Student(3, "Carol White", "C");
18
19            view.DisplayStudentList(students);
20
21            // Test 9: Update grade for first student
22            Console.WriteLine("9. Updating First Student's Grade:");
23            StudentController controller1 = new StudentController(students[0], view);
24            controller1.UpdateGrade("A");
25            controller1.DisplayStudent();
26
27            Console.WriteLine("\n=== MVC Pattern Demo Complete ===");
28            Console.WriteLine("Key Benefits:");
29            Console.WriteLine("  Separation of Concerns: Model, View, Controller are separate");
30            Console.WriteLine("  Easy Maintenance: Changes to UI don't affect data logic");
31            Console.WriteLine("  Reusability: View can be reused with different controllers");
32            Console.WriteLine("  Parallel Development: Different teams can work on MVC components");
33        }
34    }
35 }
```

PS D:\Cognizant Hands-On\MVCPatternExample> dotnet run

Easy Maintenance: Changes to UI don't affect data logic  
Reusability: View can be reused with different controllers  
Parallel Development: Different teams can work on MVC components

PS D:\Cognizant Hands-On\MVCPatternExample>



File Edit Selection View Go Run Terminal Help

MVCPatternExample

EXPLORER

- MVCPATTERNEXAMPLE
  - bin
  - obj
  - MVCPatternExample.csproj
  - Program.cs
  - Student.cs
  - StudentController.cs
  - StudentView.cs

MVCPatternExample.csproj

```
1 <Project Sdk="Microsoft.NET.Sdk">
2
3   <PropertyGroup>
4     <OutputType>Exe</OutputType>
5     <TargetFramework>net8.0</TargetFramework>
6     <ImplicitUsings>enable</ImplicitUsings>
7     <Nullable>enable</Nullable>
8   </PropertyGroup>
9
10 </Project>
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK

PS D:\Cognizant Hands-On\MVCPatternExample> dotnet run

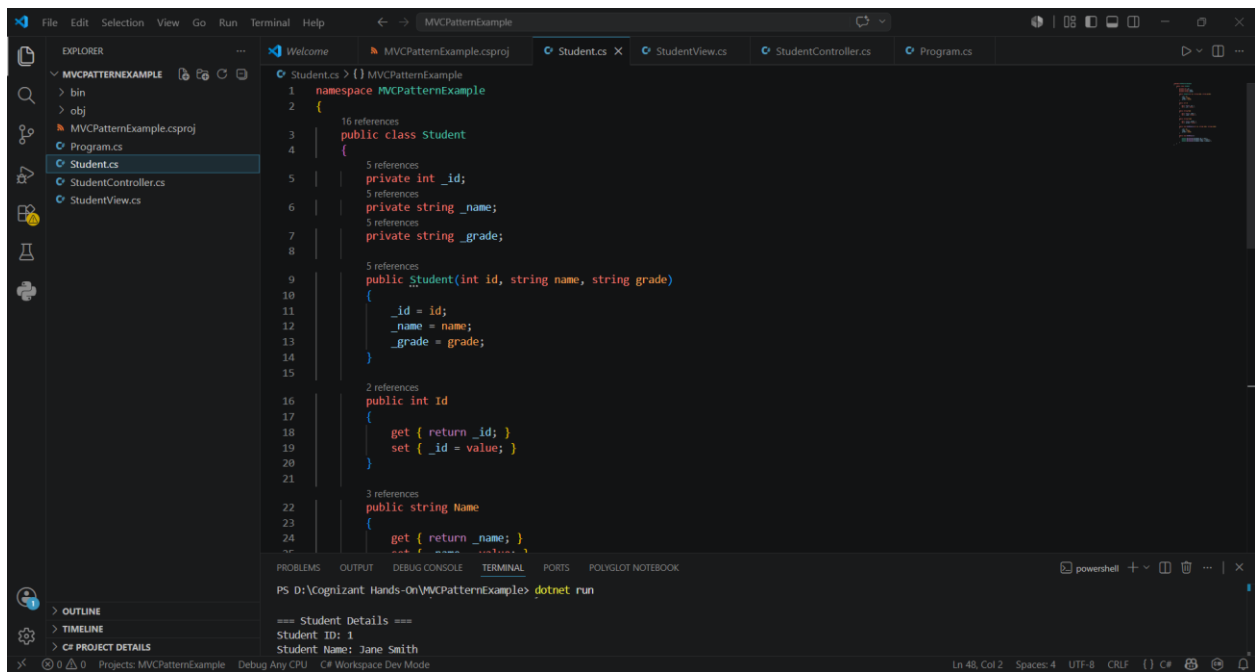
```
=== Student Details ===
Student ID: 1
Student Name: Jane Smith
Student Grade: B
=====

3. updating only grade:
SUCCESS: Grade updated to A

=== Student Details ===
Student ID: 1
Student Name: Jane Smith
Student Grade: A
=====

4. updating Only Name:
SUCCESS: Name updated to Robert Johnson
```

Ln 10, Col 11 Spaces: 4 UTF-8 CRLF {} XML



File Edit Selection View Go Run Terminal Help

MVCPatternExample

EXPLORER

- MVCPATTERNEXAMPLE
  - bin
  - obj
  - MVCPatternExample.csproj
  - Program.cs
  - Student.cs
  - StudentController.cs
  - StudentView.cs

Student.cs

```
1 namespace MVCPatternExample
2 {
3     16 references
4     public class Student
5     {
6         5 references
7         private int _id;
8         5 references
9         private string _name;
10        5 references
11        private string _grade;
12
13        5 references
14        public Student(int id, string name, string grade)
15        {
16            _id = id;
17            _name = name;
18            _grade = grade;
19        }
20
21        2 references
22        public int Id
23        {
24            get { return _id; }
25            set { _id = value; }
26        }
27
28        3 references
29        public string Name
30        {
31            get { return _name; }
32        }
33    }
34 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK

PS D:\Cognizant Hands-On\MVCPatternExample> dotnet run

```
=== Student Details ===
Student ID: 1
Student Name: Jane Smith
```

Ln 40, Col 2 Spaces: 4 UTF-8 CRLF {} C#

## Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MVCPatternExample> dotnet build
Restore complete (0.4s)
MVCPatternExample net8.0 succeeded (2.2s) → bin\Debug\net8.0\MVCPatternExample.dll

Build succeeded in 3.1s
PS D:\Cognizant Hands-On\MVCPatternExample> dotnet run
=== MVC Pattern Demo - Student Record Management ===

1. Creating Initial Student:

=== Student Details ===
Student ID: 1
Student Name: John Doe
Student Grade: A
=====

2. Updating Student Details:
SUCCESS: Student details updated successfully

=== Student Details ===
Student ID: 1
Student Name: Jane Smith
Student Grade: B
=====

3. Updating Only Grade:
SUCCESS: Grade updated to A

=== Student Details ===
Student ID: 1
Student Name: Jane Smith
Student Grade: A
=====
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\MVCPatternExample> dotnet run

4. Updating Only Name:
SUCCESS: Name updated to Robert Johnson

=== Student Details ===
Student ID: 1
Student Name: Robert Johnson
Student Grade: A
=====

5. Testing Invalid Grade:
ERROR: Invalid grade. Must be A, B, C, D, or F
6. Testing Empty Name:
ERROR: Name cannot be empty
7. Creating New Student:

=== Student Details ===
Student ID: 2
Student Name: Emily Davis
Student Grade: C
=====

8. Managing Multiple Students:

=== Student List ===
Total Students: 3
-----
ID: 1, Name: Alice Brown, Grade: A
ID: 2, Name: Bob Wilson, Grade: B
ID: 3, Name: Carol White, Grade: C
-----
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POLYGLOT NOTEBOOK
powershell + - [ ] [ ] ... | X

PS D:\Cognizant Hands-On\MVCPatternExample> dotnet run

=== Student List ===
Total Students: 3
-----
ID: 1, Name: Alice Brown, Grade: A
ID: 2, Name: Bob Wilson, Grade: B
ID: 3, Name: Carol White, Grade: C
-----

9. Updating First Student's Grade:
SUCCESS: Grade updated to A

=== Student Details ===
Student ID: 1
Student Name: Alice Brown
Student Grade: A
=====

=== MVC Pattern Demo Complete ===
Key Benefits:
  Separation of Concerns: Model, View, Controller are separate
  Easy Maintenance: Changes to UI don't affect data logic
  Reusability: View can be reused with different controllers
  Parallel Development: Different teams can work on MVC components
PS D:\Cognizant Hands-On\MVCPatternExample>
```